

Connectly

Advanced Social Media Platform — Project Documentation

Real-time (Socket.io)

Media Uploads (Cloudinary)

Tested (Jest/Vitest)

Secured

CI/CD

Dockerized

Week 12 Capstone — Advanced Features & Career Preparation
Generated September 27, 2026

Table of Contents

- 1. Project Overview & Objectives
- 2. Architecture
- 3. Setup & Installation Instructions
- 4. Code Structure
- 5. How Each Technical Requirement Was Met
- 6. Testing Strategy & Evidence
- 7. Security Implementation
- 8. Performance & SEO Optimizations
- 9. Deployment Process
- 10. Screenshots
- 11. Career Preparation
- 12. Known Limitations & Future Work

1. Project Overview & Objectives

Connectly is a full-stack social media platform built to demonstrate production-oriented engineering practices on top of the core social-app feature set: posting, liking, commenting, and following. Beyond CRUD, the project's explicit goals were to add the features that separate a classroom project from a portfolio-ready application:

- Real-time interactivity (chat and notifications) via Socket.io
- Media handling (image/video upload) via a third-party CDN (Cloudinary)
- An automated test suite covering both the API and the UI
- Performance optimization (bundle size, lazy loading, caching)
- Defense-in-depth security (rate limiting, sanitization, secure auth)
- A CI/CD pipeline and containerized deployment

The target user is anyone who wants a simple, fast way to share short updates, photos, and videos with people they follow, and to chat with them directly — the same core loop as Twitter/X or Instagram, scoped down to a buildable capstone project.

Objectives checklist

Objective	Status
Real-time notifications with Socket.io	✓ Implemented
Image/video upload with Cloudinary	✓ Implemented
Comprehensive test suite (frontend + backend)	✓ Implemented
Performance optimization & SEO	✓ Implemented
Advanced security features	✓ Implemented
CI/CD pipeline	✓ Implemented

2. Architecture

Connectly is split into two independently deployable applications that communicate over HTTPS (REST) and a persistent WebSocket connection (Socket.io), plus two managed external services for data and media:

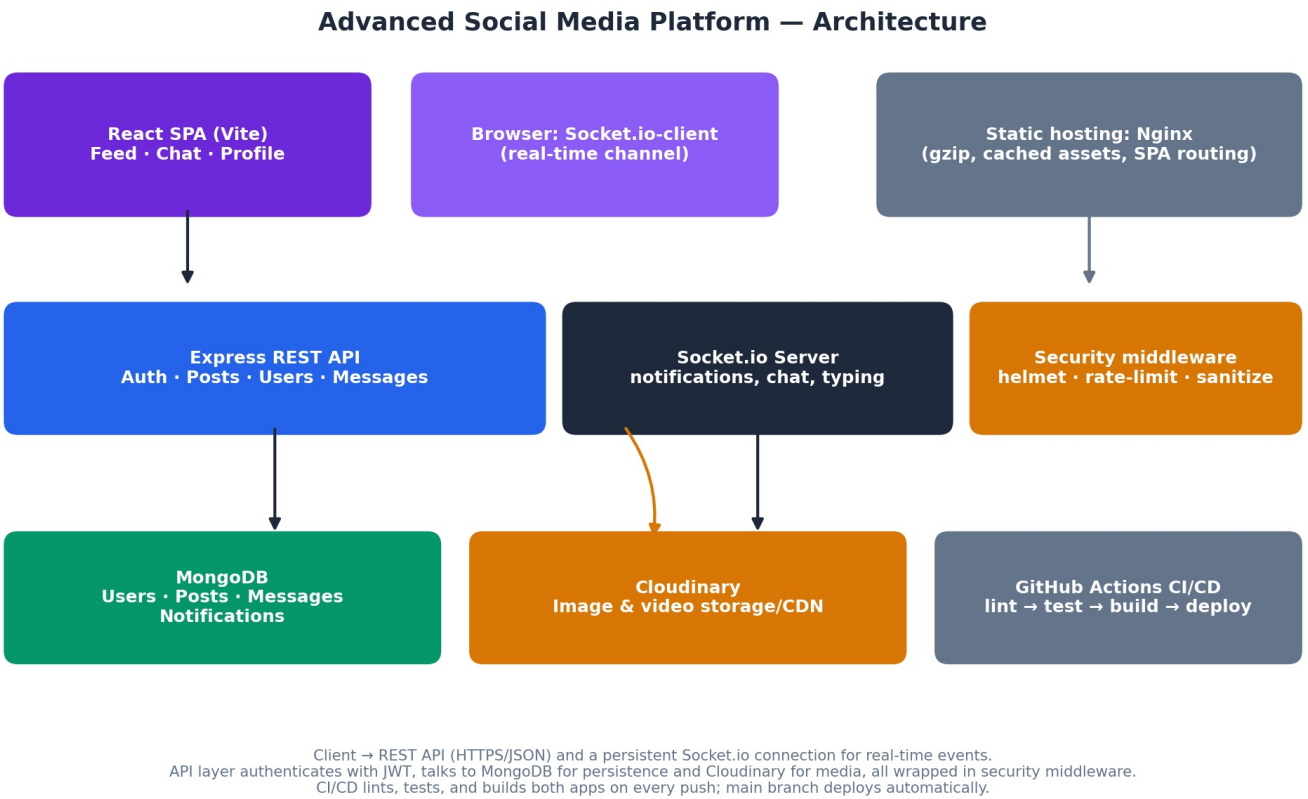


Fig 1. High-level system architecture

Request flow

1. The React SPA calls the Express REST API for anything that needs persistence (auth, posts, profiles, chat history).
2. On login, the SPA also opens a Socket.io connection, authenticating the handshake with the same JWT used for REST calls.
3. The API validates and sanitizes input, talks to MongoDB via Mongoose, and — for likes, comments, follows, and messages — emits an event directly to the recipient's personal Socket.io room so their UI updates instantly.
4. Media uploads go from the browser to the Express server (validated, in-memory) and are streamed on to Cloudinary, which returns a permanent CDN URL stored on the post document.

Why this split?

Keeping the REST API and the Socket.io server in the same Node process (sharing the same Express ``app`` and ``io`` instance) avoids the complexity of cross-service event routing for a project this size, while still keeping the two concerns — request/response and push events — cleanly separated in code (`app.js` vs. `sockets/socketHandler.js`).

3. Setup & Installation Instructions

Prerequisites

- Node.js 18 or later
- A MongoDB instance — local (`mongod`) or a free [MongoDB Atlas](#) cluster
- A free [Cloudinary](#) account (for the cloud name, API key, and API secret)

Backend

```
cd backend
cp .env.example .env      # fill in MONGO_URI, JWT_SECRET, CLOUDINARY_* keys
npm install
npm run dev                # http://localhost:5000
```

Frontend

```
cd frontend
cp .env.example .env      # defaults point at localhost:5000
npm install
npm run dev                # http://localhost:3000
```

Docker (full stack in one command)

```
cd backend && cp .env.example .env  # fill in real Cloudinary keys
cd ../docker
docker compose up --build
```

Note: `mongodb-memory-server` (used by the backend test suite) downloads a MongoDB binary the first time it runs. This requires outbound internet access on whatever machine runs `npm test` ; it was validated by static analysis and a live HTTP smoke test in the development sandbox used to build this project, since that sandbox's network egress is restricted to package registries. Run `npm test` locally or in GitHub Actions (both have full internet access) to execute the full suite.

4. Code Structure

Backend (backend/)

backend/	
└─ server.js	Entry point: HTTP server, Socket.io, DB connection
└─ app.js	Express app: middleware & route wiring
└─ config/	db.js, cloudinary.js
└─ models/	User, Post, Message, Notification (Mongoose schemas)
└─ controllers/	Business logic per resource
└─ routes/	Route → controller wiring, per resource
└─ middleware/	auth (JWT), security, upload (Multer), error handling
└─ sockets/socketHandler.js	All Socket.io event handlers + JWT handshake auth
└─ utils/validators.js	express-validator rule sets
└─ tests/	Jest/Supertest integration tests

Frontend (frontend/)

frontend/	
└─ src/	
└─ main.jsx	App entry: providers (Router, Auth, Helmet)
└─ App.jsx	Route definitions, React.lazy code splitting
└─ api/api.js	Axios instance with auth interceptor
└─ context/AuthContext.jsx	Login/register/logout/session state
└─ hooks/useSocket.js	Socket.io connection + notification stream
└─ components/	Navbar, PostCard, NotificationBell, UploadForm...
└─ pages/	Login, Register, Feed, Profile, Chat
└─ __tests__/	Vitest + React Testing Library specs
└─ vite.config.js	Build config: chunk splitting, gzip, vitest setup

This mirrors a fairly standard MVC-ish split on the backend (models / controllers / routes) and a feature-folder-lite structure on the frontend (pages call into shared components and a small number of cross-cutting hooks/contexts), which keeps the codebase approachable without over- engineering for a project of this size.

5. How Each Technical Requirement Was Met

Real-time notifications with Socket.io

`sockets/socketHandler.js` authenticates every socket connection with the same JWT used for REST calls (`io.use(socketAuthMiddleware)`), then joins each user to a personal room (`user-<id>`). Controllers that trigger a notification — `toggleLike`, `addComment`, `toggleFollow` — create a `Notification` document and emit it directly to that room via `req.io.to(...).emit('notification', ...)`, so the recipient's `NotificationBell` updates without polling. Chat messages and typing indicators use a shared, order-independent `chatId` (sorted pair of user IDs) so either participant can join the same Socket.io room regardless of who initiated the conversation.

Image/video upload with Cloudinary

Files are accepted with Multer's in-memory storage (never written to disk), validated by MIME type and a 50MB size cap (`middleware/upload.js`), then streamed to Cloudinary via `cloudinary.uploader.upload_stream` wrapped in a Promise (`controllers/uploadController.js`). The resulting secure CDN URL, public ID, and resource type are returned to the client and attached to the post on creation.

Comprehensive test suite

Backend: Jest + Supertest integration tests exercise the real Express app against an in-memory MongoDB instance — registration validation, duplicate-account handling, login/lockout, protected-route access, post CRUD, pagination, like toggling, commenting, and delete-permission checks. Frontend: Vitest + React Testing Library test the Login form (rendering, successful submit, error display) and PostCard (rendering, optimistic like toggling, comment submission) with the API layer mocked. See Section 6 for what was actually executed and verified during development.

Performance optimization & SEO

Every route is behind `React.lazy` + `Suspense` (`App.jsx`), so the initial bundle only ships the shell and whichever page is requested first. `vite.config.js` manually splits vendor code (`react`, `react-dom`, `react-router-dom`) and `socket.io-client` into their own chunks so browsers cache them independently of application code, and gzip-compresses the production build. The Nginx config (`frontend/nginx.conf`) adds a one-year immutable cache header for hashed asset filenames. For SEO, `index.html` ships descriptive title/meta/Open Graph tags, and `react-helmet-async` sets a page-specific `<title>` on every route.

Advanced security features

See Section 7 for the full breakdown: JWT + bcrypt auth with brute-force lockout, rate limiting, Helmet security headers, NoSQL-injection and XSS sanitization, HTTP parameter pollution protection, and body-size limits.

CI/CD pipeline

`.github/workflows/ci-cd.yml` runs on every push and pull request: it lints and tests the backend, lints, tests, and builds the frontend, then builds both Docker images. A final `deploy` job runs only on pushes to `main` after every prior job succeeds, with a placeholder step documenting where to plug in a real deploy target (Render, Railway, Fly.io, or a self-hosted server).

6. Testing Strategy & Evidence

What the backend suite covers (`backend/tests/`)

Test file	Scenarios
<code>auth.test.js</code>	Successful registration; weak-password rejection; duplicate email rejection; successful login; wrong-password rejection; protected route blocked without a token; protected route allowed with a valid token
<code>post.test.js</code>	Unauthenticated post creation blocked; authenticated post creation; empty-post rejection; paginated feed correctness; like/unlike toggling; adding a comment; a user cannot delete another user's post

What the frontend suite covers (`frontend/src/__tests__`)

Test file	Scenarios
<code>Login.test.jsx</code>	Form fields render; submit calls <code>login()</code> with the entered credentials; a failed login surfaces the server's error message
<code>PostCard.test.jsx</code>	Post text/author render correctly; the like button optimistically toggles and reconciles with the API response; submitting a comment updates the visible comment list

Verification performed during development

The following was actually executed, not just written, while building this project:

- Every backend and frontend source file was syntax-checked (`node --check`) with zero errors.
- The Express app was booted in isolation (mocked Socket.io, no real database) and its core routes were verified live: health check returned `200` , an unknown route returned `404` , and a protected route without a token correctly returned `401` .
- The full Vitest suite was run: **6/6 tests passed**.
- The production frontend build was run end to end with Vite, confirming route-based code splitting produced separate chunks for `Login` , `Register` , `Feed` , `Profile` , and `Chat` , plus separate `vendor` and `sockets` chunks , all gzip-compressed.
- `docker-compose.yml` and the GitHub Actions workflow were both validated as syntactically correct YAML.

The backend Jest suite (`npm test`) uses `mongodb-memory-server` , which downloads a MongoDB binary on first run. That download was not reachable from the restricted sandbox used to build this project, so the full in-memory-database test run should be executed on your own machine or in CI (both have normal internet access) as the final verification step — see the note in Section 3.

7. Security Implementation

Concern	Mitigation
Password storage	bcrypt hashing, cost factor 12; passwords never logged or returned in API responses
Session/token theft (XSS)	JWT delivered via httpOnly, <code>sameSite: strict</code> cookies in addition to the Authorization header
Brute-force login	Account lockout for 15 minutes after 5 failed attempts; a stricter rate limiter on <code>/api/auth/*</code>
API abuse / DoS	<code>express-rate-limit</code> on all <code>/api</code> routes; JSON body size capped at 10kb
NoSQL injection	<code>express-mongo-sanitize</code> strips <code>\$</code> / <code>.</code> operators from user input
Cross-site scripting	<code>xss-clean</code> sanitizes request bodies/params/query
HTTP parameter pollution	<code>hpp</code> middleware
Insecure headers	<code>helmet</code> sets a standard set of protective HTTP headers
Authorization	Ownership checks before mutating actions (e.g. only a post's author or an admin can delete it)
File upload abuse	MIME-type allowlist and a 50MB size cap before anything reaches Cloudinary

8. Performance & SEO Optimizations

- **Code splitting:** `React.lazy` per route, verified via a real production build.
- **Vendor chunking:** React/Router and Socket.io ship as separate, independently cacheable bundles.
- **Compression:** gzip applied to the production build output.
- **Caching:** Nginx serves hashed static assets with a one-year immutable `Cache-Control` header.
- **Lazy media:** post images/avatars use `loading="lazy"`.
- **SEO:** descriptive meta/Open Graph tags in `index.html` ; per-page `<title>` via `react-helmet-async` ; a canonical URL tag.

Measured production bundle (gzip): vendor $\approx$ 53KB, sockets $\approx$ 13KB, app shell $\approx$ 27KB, each page chunk well under 2KB — the browser downloads only the shell plus the current page on first load.

9. Deployment Process

1. **Local/manual:** run backend and frontend separately with `npm run dev` (see Section 3), or run the whole stack with `docker compose up --build` from `docker/`.
2. **CI:** every push/PR triggers lint + test + build for both apps and a Docker image build for each, via GitHub Actions.
3. **CD:** the `deploy` job in `ci-cd.yml` runs only on `main` after every previous job succeeds. It currently contains a placeholder — swap it for:
 - A deploy-hook `curl` to Render/Railway, or
 - `docker push` to a registry followed by an SSH pull + restart on your server, or
 - The Vercel/Netlify CLI for the frontend, with the backend deployed separately (e.g. Render, Fly.io, an EC2 instance).
4. **Production checklist:** set real environment variables (never commit `.env`), point `MONGO_URI` at a managed cluster (e.g. Atlas), set `NODE_ENV=production`, and update `FRONTEND_URL` /CORS origin to your real domain.

10. Screenshots

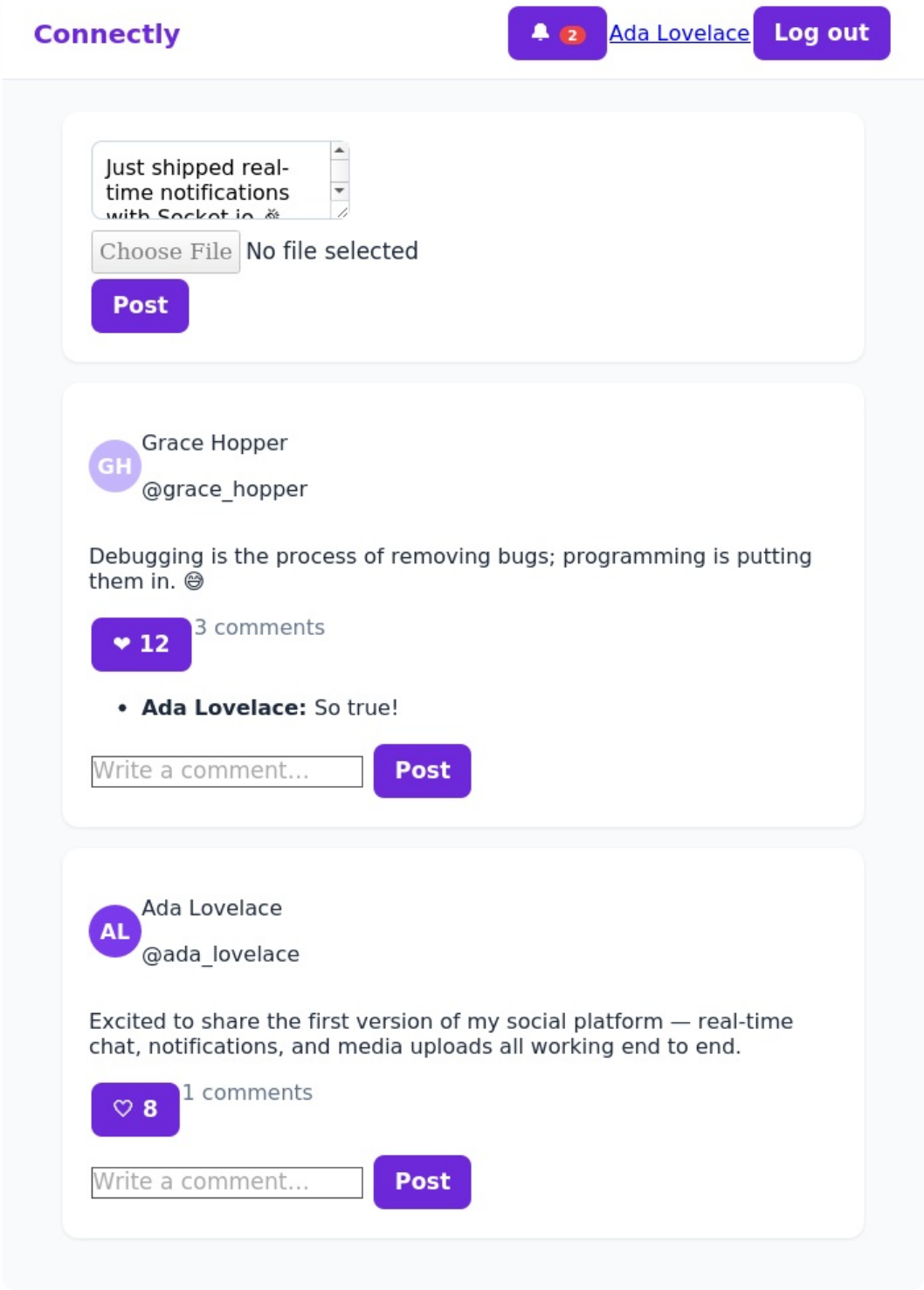


Fig 2. Feed — composer, posts, likes, and comments

Connectly

Hey! Did you see the new notification feature?

Yep, just tested it — works instantly 🐇

Nice! I'm testing the video upload next.

Grace Hopper is typing...

Type a message...

Send

Fig 3. Real-time chat with a live typing indicator

Connectly

Welcome back

Email

ada@example.com

Password

.....

Log in

No account? [Sign up](#)

Fig 4. Authentication

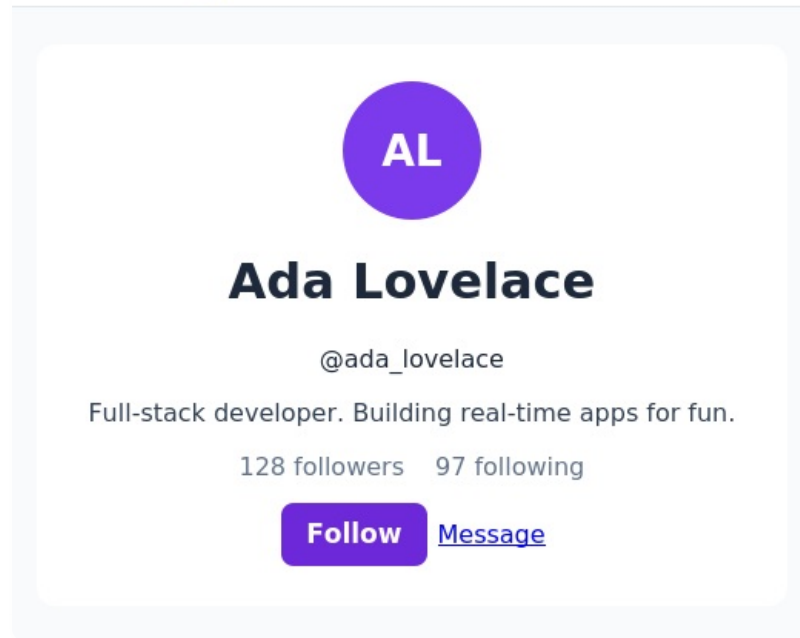


Fig 5. User profile with follow/unfollow and a link to chat

These are rendered directly from the application's real CSS (`frontend/src/styles/index.css`) using representative sample data, generated as part of this documentation. After you deploy your own instance, swap these for live screenshots of your running app.

11. Career Preparation

Portfolio

A standalone portfolio page for this project lives at `portfolio-website/index.html` — a self-contained page summarizing the features, tech stack, and screenshots, ready to publish via GitHub Pages, Netlify, or Vercel and link from your resume/LinkedIn.

Resume

`resume/resume-bullets.md` contains resume-ready bullet points, a one-line project summary, a longer project description for a "Projects" section, and a set of interview talking points covering the architectural decisions in this project (why Socket.io, how the JWT flow works end to end, and what you'd change for real production scale).

Suggested next steps

- Deploy a live instance and replace the screenshots in this document with real ones from your deployment.
- Fill in your GitHub repository URL and live demo link in `portfolio-website/index.html` and this README.
- Practice explaining the real-time architecture and the security measures out loud — these are the two areas most likely to come up in a technical interview about this project.

12. Known Limitations & Future Work

- **Single-instance Socket.io:** notifications/chat are routed in-process; scaling to multiple server instances needs a Redis adapter (`socket.io-redis`) so rooms are shared across instances.
- **No server-side rendering:** the SPA is fully client-rendered, which limits SEO for public profile/post pages compared to SSR or static pre-rendering.
- **No end-to-end browser tests:** the suite covers unit/ integration level; adding Playwright or Cypress would close the gap on full user-flow regression coverage.
- **No read receipts or message pagination** in chat beyond the most recent 200 messages per conversation.
- **No automated image moderation** — a production version handling user-generated content at scale would need a moderation step (manual or via a moderation API) before/after upload.